REPORT

To-Do List Application Using Python

Internship Program
Submitted By: Christo Rajesh
Organization: DecodeLabs
Technology Used: Python Programming

Introduction

Task management is an essential part of personal and professional productivity. A To-Do List application helps users organize and track their daily activities efficiently. This project was developed using Python to demonstrate fundamental programming concepts such as lists, loops, conditional statements, and user input handling.

The application allows users to add tasks and view all stored tasks through a simple command-line interface. The project serves as an introduction to data management and demonstrates how multiple pieces of information can be stored and managed within a single program.

Objective

The primary objectives of this project are:

- To understand Python list operations.
- To learn how to store multiple items in a single variable.
- To implement user interaction using input and output functions.
- To practice loops and conditional statements.
- To build a simple task management system.

Problem Statement

Managing multiple tasks manually can become difficult and inefficient. The objective of this project is to create a digital task manager where users can:

- Add new tasks.
- View existing tasks.
- Exit the application when desired.

The program should store tasks dynamically during execution and display them whenever requested.

System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 2 GB RAM

Software Requirements

- Python 3.x
- Visual Studio Code / PyCharm / IDLE

Methodology

The project follows the IPO (Input-Process-Output) model.

Input

- User enters task details.
- User selects menu options.

Process

- Tasks are stored in a Python list.
- The program processes user choices using conditional statements.

Output

- Displays confirmation messages.
- Displays the complete task list.

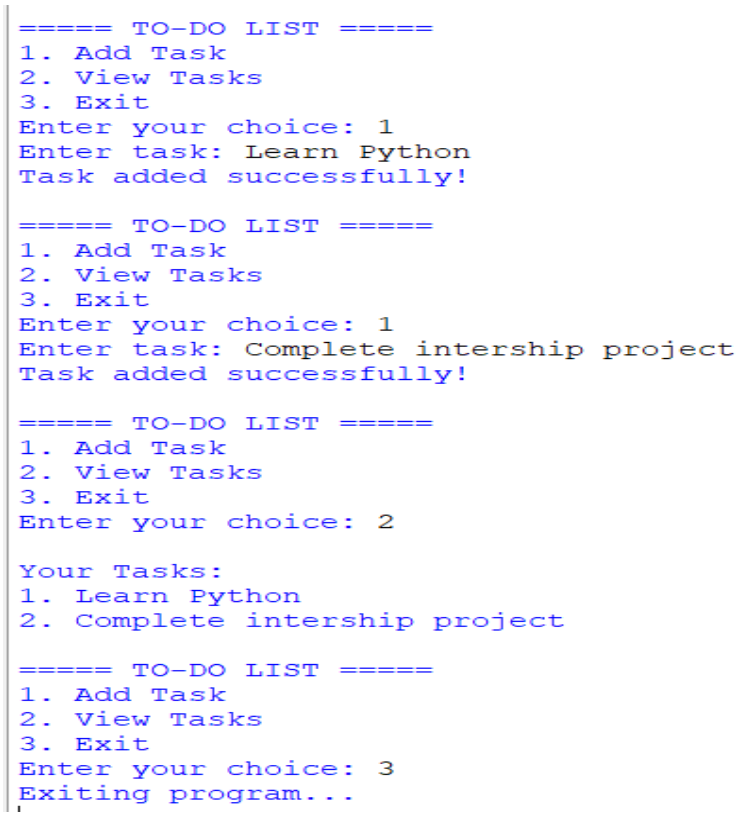
Algorithm

1. Create an empty list named tasks.
2. Display a menu with available options.
3. Accept user choice.
4. If the user selects "Add Task":
   - Accept task input.
   - Append task to the list.
5. If the user selects "View Tasks":
   - Display all tasks using a loop.
6. If the user selects "Exit":
   - Terminate the program.
7. Repeat until the user exits.

Source Code

```
tasks = []

while True:
    print("\n===== TO-DO LIST =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        task = input("Enter task: ")
        tasks.append(task)
        print("Task added successfully!")

    elif choice == "2":
        if len(tasks) == 0:
            print("No tasks available.")
        else:
            print("\nYour Tasks:")
            for i, task in enumerate(tasks, start=1):
                print(f"{i}. {task}")

    elif choice == "3":
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Try again.")
```

Output Screenshots



```
===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Exit
Enter your choice: 1
Enter task: Learn Python
Task added successfully!

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Exit
Enter your choice: 1
Enter task: Complete intership project
Task added successfully!

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Exit
Enter your choice: 2

Your Tasks:
1. Learn Python
2. Complete intership project

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Exit
Enter your choice: 3
Exiting program...
```

Features

- Simple and user-friendly interface.
- Dynamic task storage using Python lists.
- Menu-driven program structure.
- Efficient task viewing using loops.
- Easy to understand and modify.

Learning Outcomes

Through this project, the following concepts were learned:

- Python Lists
- append() Function
- Loops (for and while)
- Conditional Statements (if-elif-else)
- User Input Handling
- Basic Data Management

Future Enhancements

The application can be improved by adding:

- Task deletion functionality.
- Task completion status.
- Task editing functionality.
- File storage for permanent data saving.
- Graphical User Interface (GUI).
- Database integration using SQLite or MySQL.

Conclusion

The To-Do List Application successfully demonstrates the use of Python programming fundamentals to manage and organize tasks. The project provides practical experience in handling data using lists and implementing basic CRUD-like operations. It serves as a strong foundation for learning advanced concepts such as file handling, databases, and backend development.

Submitted as part of the DecodeLabs Industrial Training Program (Batch 2026).